How AI Works in STAAD

Oct 20, 2026 · @Om Kumar Gupta

Overview

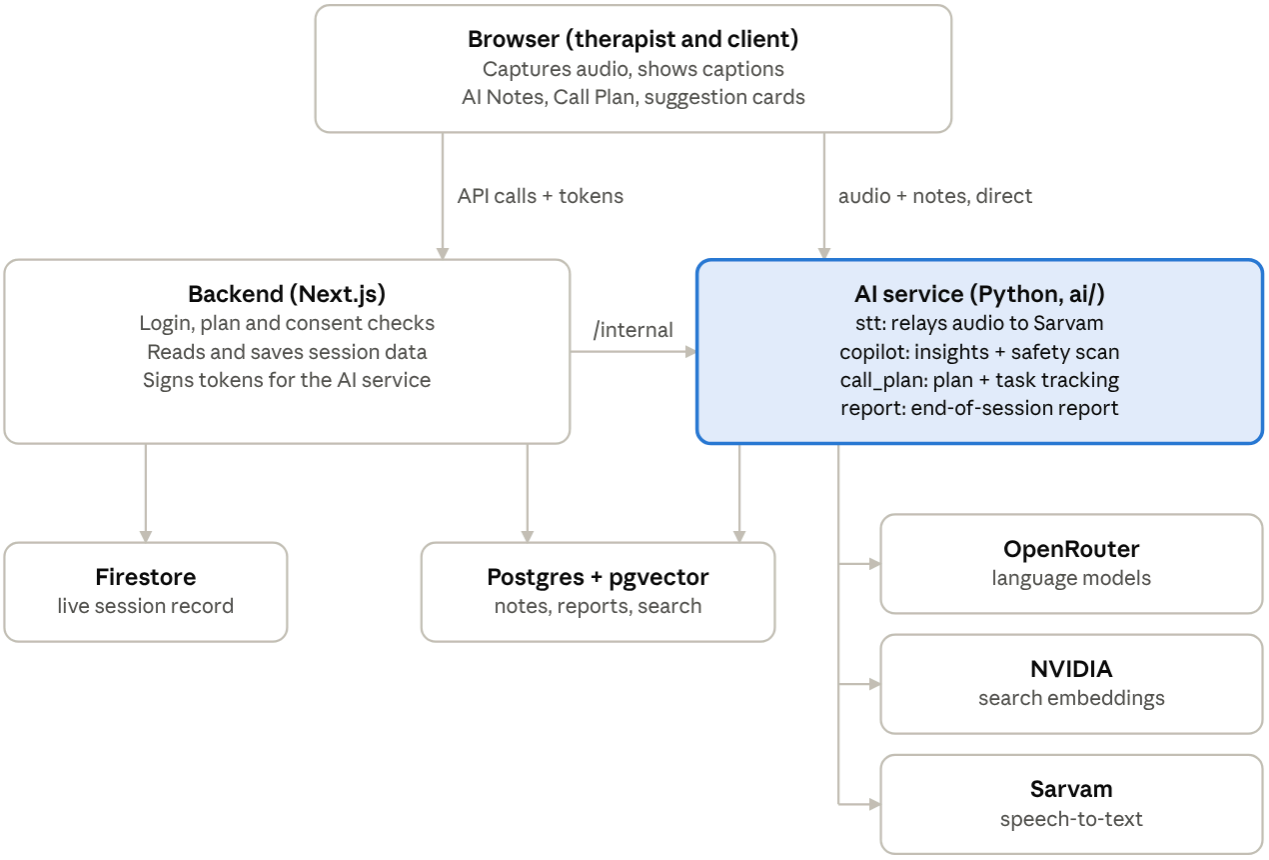
STAAD's AI listens to a therapy call (with both people's consent), turns speech into text, and uses that text to help the therapist live and after the call. It does not train or host its own model: it calls hosted models on OpenRouter for language, NVIDIA for search embeddings, and Sarvam for speech-to-text.

All AI work runs in one Python service (`ai/`). The backend decides who may use AI and stores the results; the browser shows them.

Feature	What the therapist gets	When it runs	Model / service
Speech-to-text	Live transcript and captions	Whole call, once both consent	Sarvam
AI insights (copilot)	Emotions, summary, next steps, a module, a risk level	Every 3 min automatically (if new speech), or on **Analyse**	OpenRouter
Safety scan	Risk level: none, watch or urgent	With every insight, in parallel	OpenRouter
Call Plan agent	Recommended modules and a task list from previous notes; tasks ticked during the call	When the therapist uploads or pastes notes	OpenRouter + NVIDIA
Session report	Written report with six sections, plus talk statistics	After the call, on request	OpenRouter

Architecture

All AI work runs in the AI service; the backend guards access



STAAD AI architecture · who calls what

The browser reaches the AI service directly only for speech-to-text and Call Plan, using tokens from the backend. AI insights and reports go through the backend, which checks the request, sends the session data to the AI service's private `/internal` endpoint, and saves the result. The AI service has no access to Firestore; Postgres holds both app data and the search tables.

Live speech-to-text

The therapist's browser transcribes the call; everything else the AI does is built on that transcript. It starts only after both the therapist and the client accept AI assistance.

1. The therapist's browser asks the backend for a short-lived token (`/api/stt-token`).
2. It opens one stream per person's audio track to the AI service (`/sarvam-stream`). The AI service adds the Sarvam API key, which a browser can't send, and relays audio to Sarvam.

3. Each finished sentence comes back labelled therapist or client, because each stream belongs to one person.
4. The browser saves the line to the session record in Firestore (`sessions/{id}.transcript`) and publishes it as a caption to both people over LiveKit.

Captions on screen are a display setting: the therapist's **Live captions on video** switch hides them for both people, but transcription keeps running so insights, Call Plan and the report still work. Transcripts are deleted after the retention window, 24 hours by default (admin setting `retention.transcriptHours`).

AI insights and auto suggestions

The copilot reads the last 5 minutes of the call and returns a short, structured suggestion, shown to the therapist as a card over the video. It runs automatically about every 3 minutes when something new was said, or at once when the therapist presses **Analyse**.

When it runs automatically. Only in the therapist's browser, only while transcription is on, only while the call tab is in view, and never while a check is already running. The therapist can turn it off with **Auto suggestions** in the AI Assistant popup.

What the backend does first (`/api/ai-insight`):

1. Checks the caller is the session's therapist, AI insights are on in admin settings, the therapist has an active plan, and both people consented.
2. Gathers the data: the transcript, module activity and in-call notes (Firestore), and the therapist's last 3 notes on this client (Postgres).
3. Sends it to the AI service's private `/internal` endpoint.

What the AI service does (`ai/copilot`):

1. Keeps only the last 5 minutes of transcript.
2. Searches the therapist's own study material for the 3 closest passages (if any was loaded).
3. Runs two model calls in parallel: the **analysis** (emotions, summary, 2–4 next steps, one module from the 21-module list) and a separate **safety scan**.
4. Lets the safety scan decide risk: "watch" or "urgent" raises the flag even if the analysis missed it. An unreadable safety reply counts as "watch", never as safe.

What happens to the result. The backend saves it on the session, adds it to an audit trail (`copilotEvents`), logs one usage event, and records a risk flag for the admin monitor (watch or urgent; the same level is not repeated within 30 minutes). The therapist sees a

card with the summary, two next steps and a **Launch** button for the suggested module. Watch risk is amber; urgent risk is red and stays until dismissed.

With the default free model one check takes about 17–30 seconds, so a card appears shortly after the moment it describes.

Call Plan agent

The Call Plan tab turns a client's previous notes into a plan for this call: up to 3 recommended modules and 3–8 tasks, which the agent ticks off as the conversation covers them. It is the only AI feature the browser talks to directly, with a token the backend issues after the same checks as insights (`/api/call-plan-token`).

1. **Read.** The therapist uploads notes (`.pdf`, `.docx`, `.txt` or `.md`, up to 5 MB) or pastes them. The model pulls out the concerns, progress, open homework and any risk.
2. **Search.** The agent searches the STAAD module reports in Postgres (`module_report_chunks`, 325 report sections today) for the 8 best-matching sections, at most 2 per module, then adds the selection criteria and safety limits of the top 4 modules. It searches by meaning with NVIDIA embeddings, and falls back to keyword search if NVIDIA doesn't answer.
3. **Plan.** The model picks modules only from the modules this therapist may launch, grounded in those report sections, and writes the task list.
4. **Check.** The plan is validated against the catalogue; if it is unusable the model is told what was wrong and tries once more. If the notes show risk, a safety check-in becomes the first task.
5. **Track.** During the call the agent compares the task list with the transcript and returns which tasks are covered and what to do next.

Uploaded files are read in memory and never stored. The plan is saved on the session and included in the session report.

Session report

After the call the therapist generates a written report; the AI writes the narrative and the backend computes the numbers. The therapist can edit and save it afterwards.

1. The backend (`/api/session-report`) checks the caller owns the session and has an active plan.
2. It reads the session record: the full transcript, module activity, in-call notes and the call plan.

3. It computes the statistics itself, with no AI (for example each person's share of the talking), and saves them on the session now, because the transcript is deleted after 24 hours.
4. It sends the data to the AI service (`ai/report`), which asks the model for a report in English with six sections: Session Summary, Presentation & Emotional State, Activities & Engagement, Therapist Observations, Risk & Safeguarding, and Recommendations & Next Steps.
5. The backend saves the report in Postgres with the model's name, marked as AI-generated until the therapist edits it.

The prompt tells the model to use only the transcript, activity log and notes, and to say plainly when a section has too little information rather than invent facts.

Models and services

Every language task uses one free OpenRouter model by default, `nvidia/nemotron-3-ultra-550b-a55b:free` ; each feature can be switched to another model with a setting in `ai/.env` . The previous default, the free `nvidia/nemotron-3-super-120b-a12b:free` , stopped working in October 2026 (its provider returns 404).

Service	Used for	Model or detail	Setting
OpenRouter	AI insights and safety scan	Default model (above), ~17–30 s a call	`OPENROUTER_COPILOT_MODEL`
OpenRouter	Session report	Default model, up to 2,000 tokens	`OPENROUTER_MODEL`
OpenRouter	Call Plan agent	Default model	`OPENROUTER_AGENT_MODEL`
NVIDIA	Search embeddings (module reports, study material)	`nvidia/llama-nemotron-embed-vl-1b-v2` , 1,024 dimensions	`NVIDIA_API_KEY`
Sarvam	Speech-to-text	Streaming, via the AI service relay	`SARVAM_API_KEY`
Postgres + pgvector	Module reports, study material, notes, reports	`module_report_chunks` , `document_chunks`	`DATABASE_URL`

Service	Used for	Model or detail	Setting
Firestore	Live session record: transcript, insights, call plan	`sessions/{id}`	Backend only
LiveKit	Video, and captions sent to both people	Topic `lk.transcription`	Backend + browser

A faster option is the paid `nvidia/nemotron-3-super-120b-a12b`, which answered in about 3 seconds in testing. Free models can be rate-limited or withdrawn without notice.

Data, consent and security

No AI feature runs until both the therapist and the client have allowed AI assistance in the call, and the backend checks this again on every request. A developer shortcut that skips the client's consent (`NEXT_PUBLIC_STT_TEST_MODE`) is hard-blocked in production.

- **Who may use it:** only the session's own therapist, with an active plan, and only while an admin has AI insights switched on.
- **Tokens:** every connection to the AI service needs a token signed with a shared secret (`STT_TOKEN_SECRET`), tied to one session and one feature (speech-to-text, `call-plan` or `internal`), valid for 1 hour (speech-to-text) or 5–10 minutes (the others). A token for one feature can't open another. In production the AI service refuses everything if the secret is missing.
- **What leaves STAAD:** audio goes to Sarvam; transcript text, notes and module activity go to OpenRouter and from there to the chosen model's provider; short search queries and report text go to NVIDIA for embeddings.
- **What the AI service keeps:** nothing. It has no Firebase access, reads only the search tables in Postgres, and never stores uploaded files.
- **Logs:** counts, timings and error codes only, never transcript text, notes or client details.
- **Retention:** transcripts are deleted after 24 hours by default; the report statistics, insights and report are kept with the session.

The free OpenRouter models' providers may log or use prompts under their own terms. For real client data a paid model with a no-retention policy is the safer choice (see next steps).

Known limits and next steps

The biggest risk is the free model: it is slow and the last one disappeared without warning, taking every AI feature down until it was replaced.

Limit	Effect	Suggested next step
Free default model	~17–30 s per insight; can be rate-limited or withdrawn	Switch insights to the paid `nemotron-3-super` (~3 s), or add an automatic fallback model
Free models' data terms	Client data may be kept by the provider	Use a paid model with a no-retention policy, or self-host an open model
Study material has no upload screen	`document_chunks` was empty in the database checked on 20 Oct 2026, so insights don't use it	Add an upload page that calls `ai/scripts/study_material.py` logic
No saved session notes yet (same check)	Insights have no history to draw on yet	Encourage therapists to save session notes
Usage stats count automatic checks	About 15 analyses per 50-minute call in admin stats	Log automatic and manual runs as separate types
Old AI code still in the frontend	`frontend/src/lib/rag` and `lib/report` are unused copies	Remove after checking which types the UI still imports

A fine-tuned STAAD model is possible later, on top of a self-hosted model; it needs a few hundred clinician-reviewed example plans and is not needed to fix the issues above.